ADC

1. ADC Overview

ADC (Analog-to-Digital Converter) converts analog voltage into digital values, allowing the MCU to sense the continuous physical world. This experiment reads analog voltage on GPIO4 and prints the ADC raw value (0~4095) and the calculated actual voltage via serial port. Simply use `analogRead()` to complete the entire process from sampling to quantization.

Typical Application Scenarios:

Application Type	Example
Power Monitoring	Battery level detection, USB power voltage monitoring
Environmental Sensing	Photoresistor, NTC temperature sensor, soil moisture
Human-Computer Interaction	Potentiometer knob, joystick, touch slider
Industrial Measurement	Current sampling (shunt resistor), pressure sensor, liquid level detection

2. Core Concepts

2.1 ADC Resolution and Reference Voltage

ESP32-S3 SAR ADC has 12-bit resolution:

```
Output Range: 0 ~ 4095 (2^12 - 1)
Reference Voltage: 3.3V (Vref)
```

Voltage Conversion Formula:

```
voltage = (ADC_value / 4095) × 3.3V

Example: ADC = 2047 → 2047/4095 × 3.3 ≈ 1.65V (half scale)
```

2.2 `analogRead()` Internal Flow

1. Select ADC1 controller and corresponding channel
2. Configure sampling time (default ~2.5μs)
3. Start SAR (Successive Approximation Register) ADC conversion
4. Wait for conversion to complete (blocking, ~tens of microseconds)
5. Return 12-bit result from 0~4095

2.3 Input Precautions

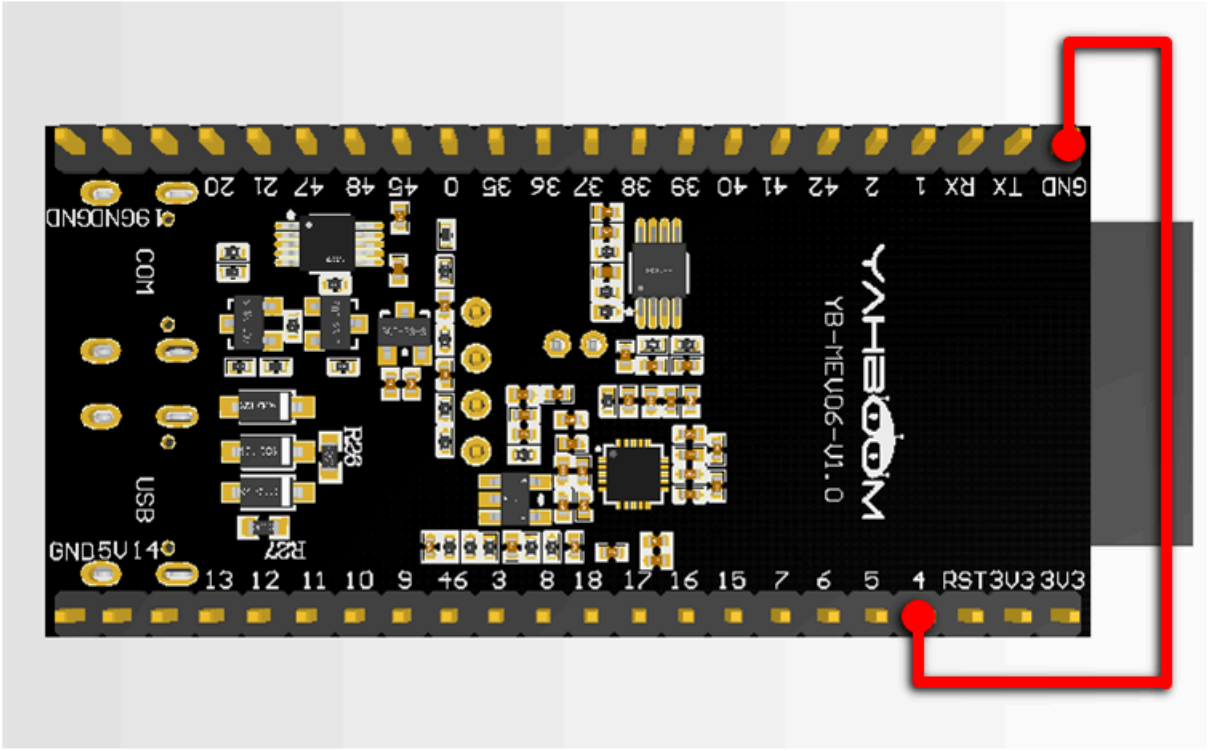
- Input range **0 ~ 3.3V**, exceeding will damage the GPIO
- To measure higher voltages (such as 5V, 12V), an external **resistor divider** circuit is required
- Floating ADC pins read random noise - this is normal, not a bug

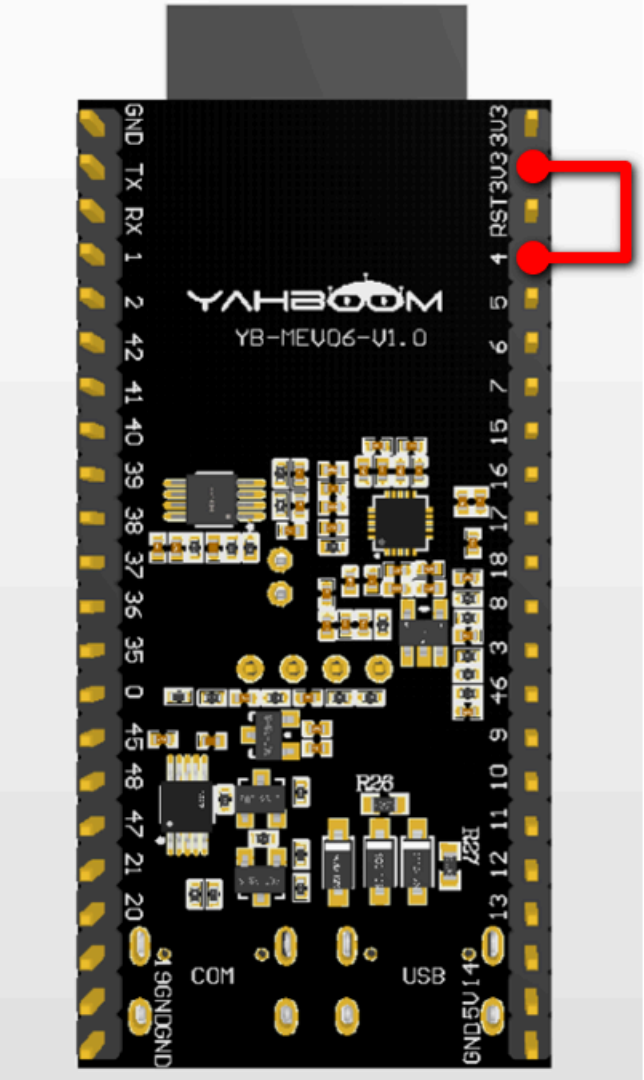
---

3. Hardware Dependencies

Pin	Peripheral	Configuration	Description
GPIO4	Analog signal source	`INPUT`	ADC1 channel 3, input range 0~3.3V

> For testing, leave it floating to observe noise values, or use a Dupont wire to connect GPIO4 to 3.3V/GND to observe full scale/zero.


GND TX RX 1 2 42 41 40 39 38 37 36 35 0 45 48 47 21 20
GND5V
COM
USB
R26
R27
YB-MEV06-V1.0
GND5V 14 13 12 11 10 9 46 3 8 18 17 16 15 7 6 5 4 RST3V3 3V3



4. Project Architecture and Flow

```
setup()
  ├─ Serial.begin(115200)      // Initialize serial port
  └─ pinMode(4, INPUT)         // Set GPIO4 as analog input

loop()
  ├─ adc_value = analogRead(4) // Read ADC value (0~4095)
  ├─ voltage = adc_value × 3.3 / 4095  // Calculate voltage
  ├─ Serial.printf(...)        // Print results
  └─ delay(1000)               // Sample once per second
```

5. Source Code

Complete source: `Source Code → Arduino → 1.Basic Peripherals → 7.ADC → 7.ADC.ino`

5.1 Pin Macro Definition

`pinMode(ADC_PIN, INPUT)` disables digital output driver and configures as high-impedance analog input.

```
#define ADC_PIN 4
```

5.2 `setup()` — Initialization

Only need to initialize serial port and pin mode. ADC peripheral is automatically configured by `analogRead`.

```
void setup() {
  Serial.begin(115200);
  pinMode(ADC_PIN, INPUT);
}
```

> Arduino's `analogRead` automatically completes ADC clock, resolution, and reference voltage configuration on the first call, no manual initialization required.

5.3 `loop()` — Sampling and Conversion

Read the raw value first, then convert to voltage using the formula. The `f` suffix in `3.3f` indicates `float` type, avoiding unnecessary `double` precision overhead.

```
void loop() {
  int adc_value = analogRead(ADC_PIN);
  float voltage = (adc_value * 3.3f) / 4095.0f;

  Serial.printf("ADC Value: %4d | Voltage: %.2f V\n", adc_value, voltage);
  delay(1000);
}
```

> `%4d` is 4-digit right-aligned formatting for neat numeric column alignment. `%.2f` keeps two decimal places.

---

6. Operation Flow

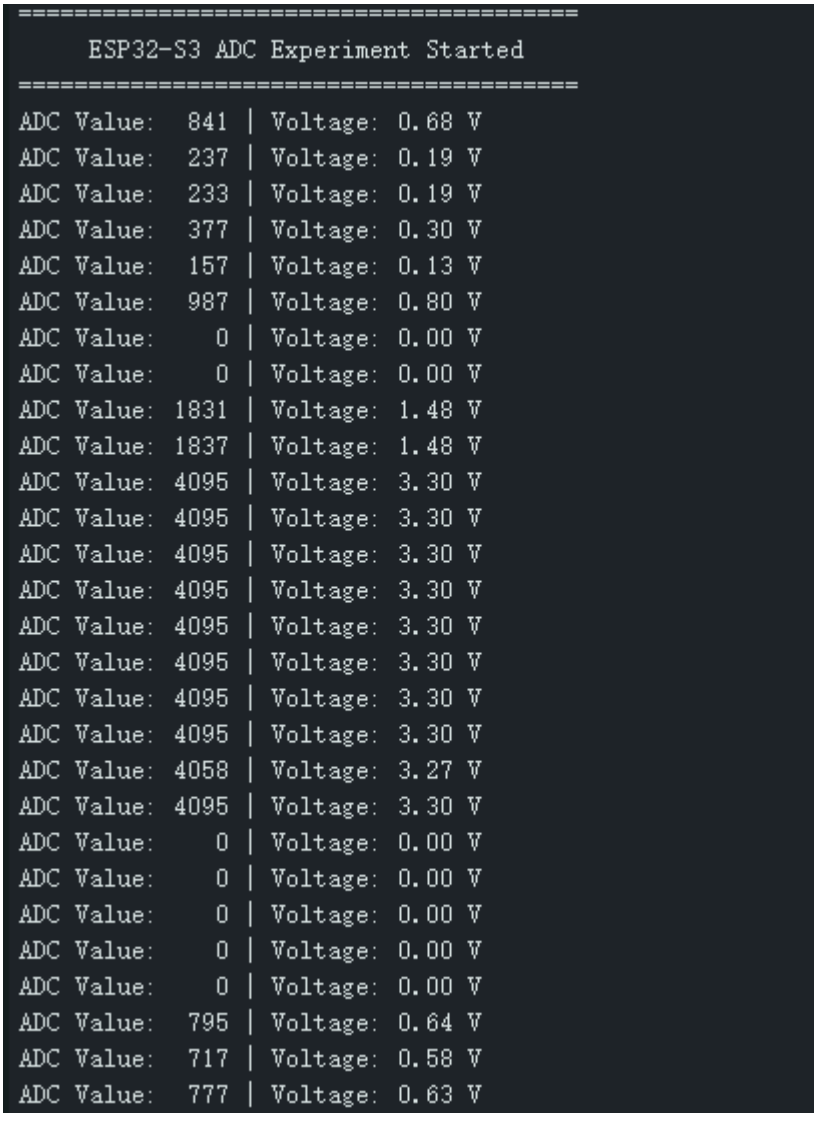
6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

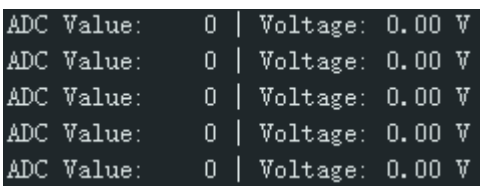
6.2 Normal Behavior

Open the **Serial Monitor** (115200 bps), one line printed per second:

```
=======================================
     ESP32-S3 ADC Experiment Started
=======================================
ADC Value:  841 | Voltage: 0.68 V
ADC Value:  237 | Voltage: 0.19 V
ADC Value:  233 | Voltage: 0.19 V
ADC Value:  377 | Voltage: 0.30 V
ADC Value:  157 | Voltage: 0.13 V
ADC Value:  987 | Voltage: 0.80 V
ADC Value:    0 | Voltage: 0.00 V
ADC Value:    0 | Voltage: 0.00 V
```



Connected to GND state:



Connected to 3V3 state:

```
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
ADC Value: 4095 | Voltage: 3.30 V
```

Floating state:

```
ADC Value:  717 | Voltage: 0.58 V
ADC Value:  777 | Voltage: 0.63 V
ADC Value: 1197 | Voltage: 0.96 V
ADC Value: 1101 | Voltage: 0.89 V
ADC Value:  613 | Voltage: 0.49 V
ADC Value:  351 | Voltage: 0.28 V
ADC Value:  295 | Voltage: 0.24 V
```

6.3 Verification Method

Operation	**Expected Result**
GPIO4 floating	Reads random noise value (normal)
GPIO4 connected to 3.3V	ADC ≈ 4095, Voltage ≈ 3.30V
GPIO4 connected to GND	ADC ≈ 0, Voltage ≈ 0.00V
Change `delay(1000)` to `delay(100)`	10 samples per second, faster value update

---

7. Common Issues

Symptom	**Cause**	**Solution**
Readings jump randomly when floating	ADC input is high impedance, pin acts like an antenna	Normal behavior, just connect a signal source
Large voltage deviation	Uncalibrated or reference voltage fluctuation	Measure actual Vref with a multimeter and substitute into formula
Readings always close to 0 or 4095	Pin shorted to GND/VDD	Check wiring